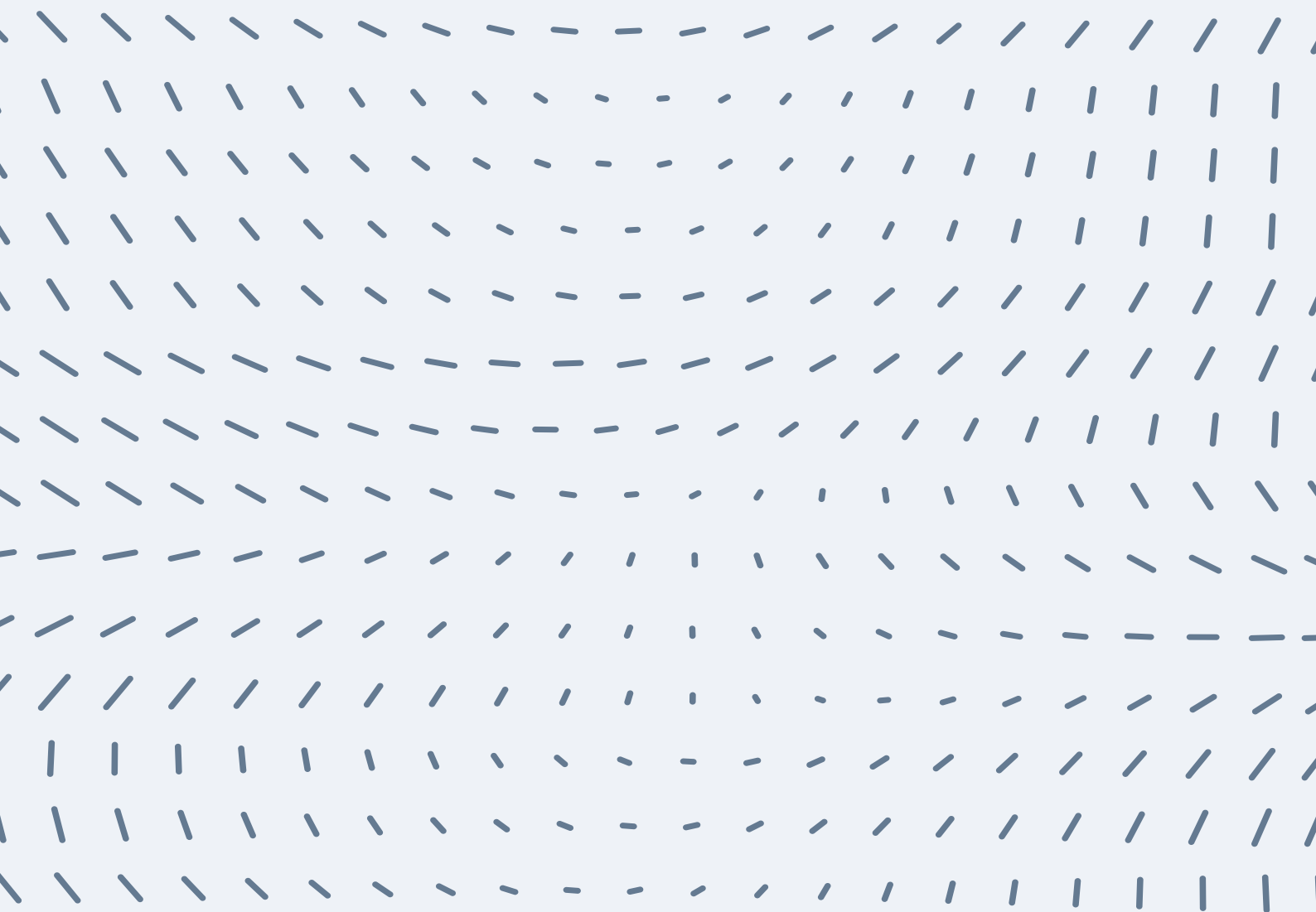


Programación orientado a objetos

Preparación Final Teórico



Programación II

(Resumen)

1. Paradigmas de Programación

Un paradigma de programación es un enfoque o estilo para resolver problemas utilizando un lenguaje de programación. Cada paradigma establece reglas y técnicas que definen cómo se deben escribir los programas.

a). Clasificación de paradigmas:

- **Paradigma Imperativo:**
 - Describe cómo resolver un problema paso a paso, especificando cada instrucción.
 - Cambia el estado del programa mediante variables.
 - Ejemplo: Lenguajes como C, Fortran.
- **Paradigma Declarativo:**
 - Especifica qué se quiere lograr, sin detallar cómo hacerlo.
 - **Subtipos:**
 - > Funcional: Basado en funciones matemáticas. Ejemplo: Haskell.
 - > Lógico: Basado en hechos y reglas. Ejemplo: Prolog.
- **Paradigma Orientado a Objetos (POO):**
 - Modela el problema como un conjunto de objetos que interactúan entre sí.
 - Ejemplo: Lenguajes como C++, Java.
- **Paradigma Estructurado:**
 - Divide el programa en estructuras lógicas claras: secuenciales, condicionales y repetitivas.

2. Programación Orientada a Objetos (POO)

> La POO es un estilo de programación basado en objetos, que son entidades que combinan datos (atributos) y comportamientos (métodos).

a). Conceptos fundamentales:

- **Clase:**
 - > Es un modelo o plantilla que define las características (atributos) y comportamientos (métodos) de un tipo de objeto.

Ejemplo:

```
class Caja {  
    private:  
        double largo, ancho, alto;  
    public:  
        double calcularVolumen() {  
            return largo * ancho * alto;  
        }  
};
```

- **Objeto:**
 - > Es una instancia de una clase. Cada objeto tiene sus propios valores para los atributos definidos en la clase.

Ejemplo:

```
Caja cajaGrande; // Objeto de la clase Caja
```

- **Atributos:**
 - > Son las variables que representan el estado del objeto.
- **Métodos:**
 - > Son funciones dentro de una clase que definen el comportamiento del objeto.

- **Encapsulamiento:**

> Protege los datos de un objeto, permitiendo acceder a ellos solo mediante métodos.

Ejemplo: Uso de palabras clave como private y public para controlar el acceso.

```
class CajaBotellas : public Caja {
private:
    int numeroBotellas;
public:
    double calcularVolumen() {
        return Caja::calcularVolumen() * 0.85; // Reutiliza código de la superclase
    }
};
```

- **Polimorfismo:**

> Permite que un método pueda tener diferentes comportamientos según el objeto que lo invoque.

Ejemplo: Uso de funciones virtuales en C++.

Los datos y funciones de una clase son llamados miembros de la clase. Las funciones miembro, a veces, también **son llamadas métodos**. A los datos miembro se los suele llamar **campos**. Cuando se define una clase, no se define un dato, sino qué significa el nombre de la clase, en qué consiste un objeto de esa clase y qué operaciones pueden realizarse

3. Constructores y Destructores

- **Constructores:**

> Son funciones especiales que inicializan los atributos de un objeto al momento de su creación.

> Tienen el mismo nombre que la clase y no retornan valor.

Ejemplo:

```
class Caja {
private:
    double largo, ancho, alto;
public:
    Caja(double l, double a, double h) : largo(l), ancho(a), alto(h) {}
};
```

- **Destructores:**

> Son funciones especiales que liberan recursos cuando el objeto deja de usarse.

> Se identifican con una tilde (~) antes del nombre de la clase.

Ejemplo:

```
~Caja() {
    // Liberar recursos
}
```

4. Punteros

> Un puntero es una variable que almacena la dirección de memoria de otra variable.

Declaración y uso:

```
int x = 10;
int *ptr = &x; // ptr almacena la dirección de x
cout << *ptr; // Desreferencia: imprime el valor de x
```

- **Operaciones comunes:**

- > Asignar direcciones:

```
ptr = &variable
```

- > Acceso al valor:

```
*ptr
```

- > Memoria dinámica:

```
int *array = new int[10]; // Reserva memoria
delete[] array;          // Libera memoria
```

5. Sobrecarga de Operadores

> Permite redefinir el comportamiento de operadores (">", "+", "-", "<") para que funcionen con objetos de una clase.

Ejemplo: Sobrecargar el operador «<» para comparar volúmenes de dos cajas:

```
class Caja {
private:
    double largo, ancho, alto;
public:
    bool operator<(const Caja& otraCaja) const {
        return (largo * ancho * alto) < (otraCaja.largo * otraCaja.ancho * otraCaja.alto);
    }
};
```

6. Plantillas (Templates)

> Permiten definir clases o funciones genéricas que funcionan con diferentes tipos de datos.

Ejemplo: Plantilla para una función que intercambia dos valores:

```
class Caja {
private:
    double largo, ancho, alto;
public:
    bool operator<(const Caja& otraCaja) const {
        return (largo * ancho * alto) < (otraCaja.largo * otraCaja.ancho * otraCaja.alto);
    }
};
```

7. Manejo de Excepciones

> Las excepciones permiten manejar errores o condiciones inesperadas durante la ejecución del programa.

Estructura básica:

```
try {
    if (condicion)
        throw "Error detectado";
} catch (const char* mensaje) {
    cout << mensaje;
}
```

8. Herencia y Polimorfismo

- **Herencia:**

> Permite que una clase derive de otra para reutilizar atributos y métodos.

```
class Animal {
public:
    virtual void hacerSonido() const {
        cout << "Sonido genérico";
    }
};

class Perro : public Animal {
public:
    void hacerSonido() const override {
        cout << "Ladrido";
    }
};
```

- **Polimorfismo:**

> Permite usar un mismo nombre para métodos en diferentes clases, ejecutando el código adecuado según el tipo de objeto.

```
Animal* animal = new Perro();
animal->hacerSonido(); // Imprime "Ladrido"
```

GUÍA PARA ANALIZAR EL CÓDIGO PASO A PASO

1. Comprensión del Enunciado

a) Lee el problema y subraya las palabras clave:

- ¿Qué elementos están involucrados? (clases, funciones, punteros, excepciones, etc.).
- ¿Qué salida o valores se te piden calcular?
- ¿Cuál es el contexto? (manejo de memoria, jerarquía de clases, polimorfismo, etc.).

b) Identifica los conceptos principales:

- ¿Involucra punteros y memoria dinámica?
- ¿Incluye herencia y funciones virtuales?
- ¿Solicita manejar excepciones o estructuras complejas?

c) Relaciona con la teoría:

- Hacer una lista de los temas específicos que aborda (ejemplo: punteros, herencia, etc.).

2. Análisis Inicial del Código

a) Declaraciones Globales

- Revisa declaraciones de variables o punteros globales, ya que pueden influir en la ejecución.

Declaración de Variables

Tipos básicos:

```
int x = 5;      // Entero
float y = 3.14; // Punto flotante
char c = 'A';   // Carácter
bool flag = true; // Booleano
```

- Inicialización múltiple:

```
int a = 1, b = 2, c = 3; // Múltiples variables enteras
```

Declaración de Constantes

- Usadas para valores que no cambian durante la ejecución.

```
const double PI = 3.14159; // Constante de punto flotante
```

Declaración de Arreglos

- Unidimensionales:

```
int arr[5] = {1, 2, 3, 4, 5}; // Arreglo de 5 elementos
```

- Multidimensionales:

```
int mat[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}; // Matriz 3x3
```

Declaración de Funciones

- Declaración y definición en la misma sección:

```
int suma(int a, int b) {  
    return a + b;  
}
```

- Declaración antes de main() y definición después:

```
int suma(int a, int b); // Declaración
```

```
int main() {  
    cout << suma(3, 4);  
    return 0;  
}
```

```
int suma(int a, int b) { // Definición  
    return a + b;  
}
```

Declaración de Estructuras

- Usadas para agrupar datos relacionados:

```
struct Punto {  
    int x;  
    int y;  
};  
  
Punto p = {10, 20};
```

Declaración de Clases

- Definición básica:

```
class Persona {  
    private:  
        string nombre;  
        int edad;  
  
    public:  
        void setNombre(string n) {  
            nombre = n;  
        }  
        string getNombre() {  
            return nombre;  
        }  
};
```

- Instancia de clase:

```
Persona p;  
p.setNombre("Carlos");  
cout << p.getNombre();
```

Declaración de Constructores y Destructores

- Constructor: Inicializa los atributos al crear un objeto.

```
class Rectangulo {  
    private:  
        int base, altura;  
  
    public:  
        Rectangulo(int b, int h) { // Constructor  
            base = b;  
            altura = h;  
        }  
};  
  
Rectangulo r(10, 20); // Instancia usando el constructor
```

- Destructor: Limpia recursos cuando el objeto es destruido.

```
~Rectangulo() {  
    cout << "Objeto destruido";  
}
```

Declaración de Punteros

- Básico:

```
int x = 10;
int* p = &x; // Puntero a la dirección de x
cout << *p; // Imprime el valor apuntado por p (10)
```

- Memoria dinámica:

```
int* p = new int(5); // Reserva memoria para un entero
delete p;           // Libera la memoria reservada
```

Declaración de Herencia

- Relación entre clases padre e hijo:

```
class Animal {
public:
    void comer() {
        cout << "El animal está comiendo";
    }
};

class Perro : public Animal {
public:
    void ladrar() {
        cout << "El perro está ladrando";
    }
};

Perro p;
p.comer(); // Llama al método heredado
p.ladrar(); // Llama al método de la clase hija
```

Declaración de Sobrecarga de Operadores

- Redefinir operadores para una clase específica:

```
class Complejo {
private:
    double real, imaginario;

public:
    Complejo(double r, double i) : real(r), imaginario(i) {}

    Complejo operator+(const Complejo& c) {
        return Complejo(real + c.real, imaginario + c.imaginario);
    }
};

Complejo c1(1.0, 2.0), c2(3.0, 4.0);
Complejo c3 = c1 + c2; // Usa el operador sobrecargado
```


Declaración de Plantillas (Templates)

- Permiten definir funciones o clases genéricas:

```
template <typename T>
T suma(T a, T b) {
    return a + b;
}

cout << suma(5, 10);    // Funciona con enteros
cout << suma(5.5, 2.3); // Funciona con flotantes
```

Manejo de Excepciones

- Declaración de bloques try-catch:

```
try {
    throw runtime_error("Error inesperado");
} catch (runtime_error& e) {
    cout << e.what();
}
```

> Si hay punteros inicializados como **NULL** o con direcciones de variables, anótalos.

b). Cabecera y Librerías

Librerías Básicas

<iostream>

- Permite la entrada y salida estándar (**cin, cout, etc.**).
- Ejemplo:

```
#include <iostream>
using namespace std;

int main() {
    int x;
    cout << "Ingresa un número: ";
    cin >> x;
    cout << "El número ingresado es: " << x << endl;
    return 0;
}
```

<cmath>

- Proporciona funciones matemáticas como raíces, potencias y trigonometría.
- Ejemplo:

```
#include <cmath>
#include <iostream>
using namespace std;

int main() {
    double x = 16.0;
    cout << "Raíz cuadrada de 16: " << sqrt(x) << endl;
    return 0;
}
```

<cstdlib>

- Funciones para manejo de números aleatorios y conversión de cadenas.

- Ejemplo:

```
#include <cstdlib>
#include <ctime>
#include <iostream>
using namespace std;

int main() {
    srand(time(0)); // Inicializa el generador de números aleatorios
    int randomNum = rand() % 100; // Número aleatorio entre 0 y 99
    cout << "Número aleatorio: " << randomNum << endl;
    return 0;
}
```

<string>

- Soporte para manejar cadenas de caracteres como objetos.

- Ejemplo:

```
#include <string>
#include <iostream>
using namespace std;

int main() {
    string nombre = "Carlos";
    cout << "Hola, " << nombre << "!" << endl;
    return 0;
}
```

<iomanip>

- Permite formatear la salida, como la precisión de números decimales.

- Ejemplo:

```
#include <iomanip>
#include <iostream>
using namespace std;

int main() {
    double pi = 3.14159;
    cout << fixed << setprecision(2) << pi << endl; // Imprime "3.14"
    return 0;
}
```

Comparación entre Programación I y II

Cabecera	Programación I	Programación II
<iostream>	Entrada y salida estándar	Entrada y salida estándar
<cmath>	Funciones matemáticas básicas	Sigue siendo útil para cálculos
<cstdlib>	Generación de números aleatorios	Uso extendido para memoria dinámica
<string>	Manejo de cadenas	Sigue usándose frecuentemente

Librerías Avanzadas

<vector>

- Permite usar vectores dinámicos en lugar de arreglos tradicionales.
- Ejemplo:

```
#include <vector>
#include <iostream>
using namespace std;

int main() {
    vector<int> numeros = {1, 2, 3, 4, 5};
    numeros.push_back(6); // Agrega un elemento al final
    for (int num : numeros) {
        cout << num << " ";
    }
    return 0;
}
```

<map> y <unordered_map>

- Permite usar mapas y diccionarios para asociar claves con valores.
- Ejemplo:

```
#include <map>
#include <iostream>
using namespace std;

int main() {
    map<string, int> edades;
    edades["Juan"] = 25;
    edades["Ana"] = 30;

    for (const auto& [nombre, edad] : edades) {
        cout << nombre << " tiene " << edad << " años." << endl;
    }
    return 0;
}
```

<fstream>

- Permite trabajar con archivos (lectura/escritura).
- Ejemplo:

```
#include <fstream>
#include <iostream>
using namespace std;

int main() {
    ofstream archivo("datos.txt");
    archivo << "Hola, archivo!";
    archivo.close();
    return 0;
}
```

<memory>

- Contiene punteros inteligentes como **shared_ptr** y **unique_ptr**.
- Ejemplo:

```
#include <memory>
#include <iostream>
using namespace std;

int main() {
    unique_ptr<int> ptr = make_unique<int>(10);
    cout << "Valor: " << *ptr << endl;
    return 0;
}
```

<exception>

- Permite manejar excepciones personalizadas.
- Ejemplo:

```
#include <exception>
#include <iostream>
using namespace std;

int main() {
    try {
        throw runtime_error("Error inesperado");
    } catch (runtime_error& e) {
        cout << e.what() << endl;
    }
    return 0;
}
```

Comparación entre Programación I y II

Cabecera	Programación I	Programación II
<vector>	Opcional (en algunos casos)	Uso extendido para listas dinámicas
<fstream>	Ocasional	Uso intensivo para manipulación de archivos
<map>	No utilizada	Muy común para estructuras asociativas
<memory>	No utilizada	Esencial para punteros inteligentes

3. Análisis de Clases y Herencia

- **Clases**

> Las clases son el núcleo de la programación orientada a objetos. Al analizar una clase, es importante observar:

Componentes de una clase

1. Atributos: Variables que describen el estado del objeto.
2. Métodos: Funciones que definen el comportamiento del objeto.
3. Modificadores de acceso:
 - > private: Accesible solo dentro de la clase.
 - > protected: Accesible en la clase y sus derivadas.
 - > public: Accesible desde cualquier parte del programa.

Análisis de la declaración de la clase

> Revisa cada elemento de la clase:

1. **Definición de la clase:**

- Nombre de la clase.
- Si utiliza template para ser genérica.

2. **Atributos:**

- Tipo de dato.
- Nivel de acceso (**private**, **public**, etc.).

3. **Métodos:**

- ¿Qué acciones realiza? ¿Cómo modifica los atributos?
- Sobrecarga de métodos (misma función con parámetros diferentes).

4. **Constructores y destructores:**

- Inicialización de atributos.
- Liberación de recursos al destruir el objeto.

5. **Sobrecarga de operadores (si existe):**

- Identifica funciones **operator+**, **operator<**, etc., que redefinan el comportamiento de operadores.

Ejemplo

```
#include <iostream>
using namespace std;

class Persona {
private:
    string nombre;
    int edad;

public:
    // Constructor
    Persona(string n, int e) : nombre(n), edad(e) {}

    // Métodos
    void mostrarDatos() {
        cout << "Nombre: " << nombre << ", Edad: " << edad << endl;
    }
};

int main() {
    Persona p1("Ana", 30);
    p1.mostrarDatos();
    return 0;
}
```

Modificadores y encapsulamiento

- Verifica qué elementos son accesibles desde fuera de la clase.
- Asegúrate de que los atributos sean private y se accedan mediante métodos (getters y setters).

• Herencia

- > En herencia, una clase (**clase derivada**) extiende las características de otra (**clase base**). Esto permite:
 - Reutilizar código.
 - Crear relaciones jerárquicas entre clases.

Relación entre clases

1. **Clase Base:**
 - Clase principal de la que derivan otras.
 - Define atributos y métodos comunes.
2. **Clase Derivada:**
 - Extiende la funcionalidad de la clase base.
 - Puede sobrescribir métodos de la clase base (polimorfismo).

Tipos de herencia

1. **Herencia pública (: public):**
 - Los miembros public y protected de la clase base son accesibles directamente en la clase derivada.
2. **Herencia protegida (: protected):**
 - Los miembros public de la clase base se vuelven protected en la clase derivada.
3. **Herencia privada (: private):**
 - Todos los miembros de la clase base se vuelven private en la clase derivada.

Ejemplo

```
#include <iostream>
using namespace std;

class Animal {
public:
    void comer() {
        cout << "Este animal está comiendo." << endl;
    }
};

class Perro : public Animal {
public:
    void ladrar() {
        cout << "El perro está ladrando." << endl;
    }
};

int main() {
    Perro p;
    p.comer(); // Heredado de Animal
    p.ladrar(); // Propio de Perro
    return 0;
}
```

- **Polimorfismo**

- > El polimorfismo permite usar una clase derivada como si fuera de la clase base, aprovechando funciones virtuales para sobrescribir comportamientos.

Funciones Virtuales

- Declaradas con la palabra clave virtual en la clase base.
- Pueden ser sobrescritas en las clases derivadas.

Ejemplo:

```
#include <iostream>
using namespace std;

class Figura {
public:
    virtual void dibujar() { // Función virtual
        cout << "Dibujando una figura genérica." << endl;
    }
};

class Circulo : public Figura {
public:
    void dibujar() override { // Sobreescritura
        cout << "Dibujando un círculo." << endl;
    }
};

int main() {
    Figura* f = new Circulo();
    f->dibujar(); // Llama a la función de Circulo
    delete f;
    return 0;
}
```

Funciones Virtuales Puras

- Declaradas con = 0 en la clase base.
- Obligan a que las clases derivadas las implementen.

Ejemplo:

```
class Animal {
public:
    virtual void hacerSonido() = 0; // Virtual pura
};

class Gato : public Animal {
public:
    void hacerSonido() override {
        cout << "Miau" << endl;
    }
};
```

4. Análisis Práctico de Clases y Herencia

> Al analizar un programa que use clases y herencia:

1. Identifica las clases base y derivadas.

2. Busca relaciones jerárquicas:

- ¿Qué clase extiende a otra? ¿Qué hereda?

3. Observa el uso de funciones virtuales:

- ¿Se usa polimorfismo? ¿Cómo se sobrescriben las funciones?

4. Comprueba encapsulamiento:

- Asegúrate de que los atributos estén protegidos o privados.

Ejemplo:

Para el siguiente código:

```
#include <iostream>
using namespace std;

class Base {
public:
    virtual void mostrar() {
        cout << "Clase Base" << endl;
    }
};

class Derivada : public Base {
public:
    void mostrar() override {
        cout << "Clase Derivada" << endl;
    }
};

int main() {
    Base* b = new Derivada();
    b->mostrar(); // Llama a la versión de Derivada
    delete b;
    return 0;
}
```

Análisis:

1. Clase Base:

- Base tiene una función virtual mostrar.

2. Clase Derivada:

- Derivada sobrescribe mostrar.

3. Relación entre clases:

- Derivada hereda de Base de forma pública.

4. Polimorfismo:

- Se usa un puntero de tipo Base para acceder al objeto de tipo Derivada.

5. Preguntas Clave

1. ¿Qué atributos y métodos tiene cada clase?
2. ¿Cómo se relacionan las clases entre sí?
3. ¿Se usa polimorfismo? ¿Cómo?
4. ¿Existen funciones virtuales o virtuales puras?
5. ¿Qué herencia (pública, privada, protegida) se utiliza?

1. Conceptos Fundamentales de POO

• Clases y Objetos

- **Clase:** Es un plano o plantilla para crear objetos. Define atributos (propiedades) y métodos (comportamientos).
- **Objeto:** Instancia de una clase. Tiene los valores definidos en sus atributos y puede ejecutar los métodos de la clase.

Ejemplo:

```
class Animal {
public:
    string nombre;
    int edad;

    void hablar() {
        cout << "Soy un animal" << endl;
    }
};

int main() {
    Animal a; // Creación de un objeto de la clase Animal
    a.nombre = "Perro";
    a.edad = 5;
    a.hablar();
    return 0;
}
```

• Encapsulamiento

- Consiste en ocultar la implementación interna de una clase y exponer solo lo necesario a través de métodos públicos. Esto se logra mediante modificadores de acceso (**private**, **protected**, **public**).
- **Ventajas:** Protege la integridad de los datos, previene acceso no autorizado y permite cambios internos sin afectar el código externo.

Ejemplo:

```
class CuentaBancaria {
private:
    double saldo;

public:
    void depositar(double cantidad) {
        saldo += cantidad;
    }

    double obtenerSaldo() {
        return saldo;
    }
};
```

Herencia

- Permite que una clase derive características de otra, promoviendo la reutilización de código.
- La clase derivada puede heredar atributos y métodos de la clase base, y además sobrescribir métodos si es necesario.

Ejemplo:

```
class Vehiculo {
public:
    void encender() {
        cout << "El vehículo está encendido." << endl;
    }
};

class Coche : public Vehiculo {
public:
    void encender() {
        cout << "El coche está encendido." << endl; //
Sobrescribe el método
    }
};
```

Polimorfismo

- Permite que un objeto se comporte de manera diferente según su tipo. Se puede lograr mediante funciones virtuales y sobrecarga de métodos.

Ejemplo:

```
class Animal {
public:
    virtual void hacerSonido() {
        cout << "Sonido de animal" << endl;
    }
};

class Gato : public Animal {
public:
    void hacerSonido() override {
        cout << "Miau" << endl; // Sobrescritura de la
función
    }
};

int main() {
    Animal* animal = new Gato();
    animal->hacerSonido(); // Llamada a
Gato::hacerSonido
    delete animal;
    return 0;
}
```

Constructores y Destructores

- **Constructor:** Método especial que se llama cuando se crea un objeto. Se utiliza para inicializar los atributos.
- **Destructor:** Método especial que se llama cuando un objeto es destruido. Se utiliza para liberar recursos dinámicos.

Ejemplo:

```
class Persona {
private:
    string nombre;
public:
    // Constructor
    Persona(string n) : nombre(n) {
        cout << "Constructor de Persona llamado." << endl;
    }

    // Destructor
    ~Persona() {
        cout << "Destructor de Persona llamado." << endl;
    }
};
```

Sobrecarga de Operadores

- Permite redefinir los operadores estándar (como +, -, =, etc.) para que funcionen con objetos de clases definidas por el usuario.

Ejemplo de sobrecarga del operador +:

```
class Complejo {
private:
    double real, imaginario;
public:
    Complejo(double r, double i) : real(r), imaginario(i) {}

    // Sobrecarga del operador "+"
    Complejo operator+(const Complejo& otro) {
        return Complejo(real + otro.real, imaginario +
otro.imaginario);
    }

    void mostrar() {
        cout << real << " + " << imaginario << "i" << endl;
    }
};
```

Herencia y Sobreescritura de Métodos

- La sobreescritura de métodos permite que las clases derivadas modifiquen la implementación de un método heredado de la clase base.
- La herencia múltiple es un caso especial donde una clase puede heredar de más de una clase base.

Ejemplo de herencia múltiple:

```
class A {
public:
    void metodoA() {
        cout << "Método de A" << endl;
    }
};

class B {
public:
    void metodoB() {
        cout << "Método de B" << endl;
    }
};

class C : public A, public B {
public:
    void metodoC() {
        cout << "Método de C" << endl;
    }
};
```

Conceptos Clave para Recordar

1. **Encapsulamiento:** Protege los datos internos de las clases.
2. **Herencia:** Crea jerarquías de clases.
3. **Polimorfismo:** Permite que una misma función se comporte de manera diferente dependiendo del objeto que la invoque.
4. **Constructores y Destructores:** Inicializa y limpia los objetos.
5. **Sobrecarga de Operadores:** Permite redefinir el comportamiento de los operadores para objetos de una clase.

Resolución de Ejercicios

- **Análisis de la Declaración de Clases**

1. **Identifica los atributos de la clase:** ¿Qué datos guarda la clase?
2. **Identifica los métodos de la clase:** ¿Qué acciones puede realizar la clase? ¿Hay métodos sobrecargados?
3. **Verifica el tipo de herencia:** ¿Es pública, privada o protegida? ¿Qué se hereda de la clase base?

- **Análisis de Polimorfismo y Sobrescritura**

1. **Busca funciones virtuales:** ¿Se utiliza polimorfismo en el código? ¿Cómo?
2. **Verifica la implementación de funciones sobrecargadas o sobrescritas:**
¿Cómo cambia el comportamiento de las clases derivadas?

- **Ejemplo de Análisis de un Ejercicio Teórico**

Suponé que te dan este código para analizar:

```
class Animal {
public:
    virtual void hacerSonido() {
        cout << "Sonido de animal" << endl;
    }
};

class Perro : public Animal {
public:
    void hacerSonido() override {
        cout << "Guau" << endl;
    }
};

int main() {
    Animal* a = new Perro();
    a->hacerSonido(); // Salida: "Guau"
    delete a;
    return 0;
}
```

Análisis:

1. **Clase base (Animal):**
 - Tiene un método virtual **hacerSonido()**.
2. **Clase derivada (Perro):**
 - Sobrescribe **hacerSonido()** con una implementación específica.
3. **Polimorfismo:**
 - Usamos un puntero de tipo Animal para apuntar a un objeto de tipo Perro, y el comportamiento es determinado en tiempo de ejecución gracias al polimorfismo.
4. **Salida: "Guau".**